

操作系统实验文档

Lab 13 - Synchronization 3

f2b6767c

Built by github-actions, at 2025-05-13 11:48:26+08:00

1. Weekly Lab Material

1.0.1 1.3 Week 13 - Synchronization 3

1.3.1 Experiment Objectives

1. Master the causes and solutions for concurrent bugs.
2. Master the necessary conditions for deadlock.
3. Master user-mode locking mechanisms.

1.3.2 Concurrent Bugs

To protect resources, we need to implement locking. However, locking is a very complex and delicate operation. Therefore, we may encounter **deadlock** if the locking order is incorrect, or **data race** might still exist if the locking method is flawed. We refer to these as concurrent bugs.

DEADLOCK

Deadlock problems are usually divided into two types: AA-deadlock and ABBA-deadlock.

AA-deadlock

AA-deadlock means you are trying to acquire a lock that you have already acquired:

```
acquire(&lk);
assert(holding(&lk));
// ...

// maybe in interrupt handler or other methods
acquire(&lk);
// <-- deadlock here.
```

Although it seems like we wouldn't make this mistake, real systems are often complex, and the control flow of functions may not be so obvious.

Imagine you are writing function `c()`, and functions `B` and `A` already exist. When `B` calls `A`, `A` acquires and releases a lock. `c` might also need to use the same lock, so you also acquire and release the lock in `c`. Then you find that you need to reuse the functionality of `A`, so you call `A` while holding the lock.

```
void A() {
    acquire(&lk);
```

```

    // ...
    release(&lk);
}

void B() {
    // ...
    A();
    // ...
}

void C() {
    acquire(&lk);
    A();
    release(&lk);
}

```

When control flow goes from `B` to `A`, it will not deadlock. However, when it goes from `C` to `A`, it will deadlock.

Also, imagine another scenario where you debugged and found a concurrent bug; to solve it, you added acquire and release code in `A`; you might think only `B` will call `A`, but in reality, there exists a call chain `C -> A`, and in this call chain, you already acquired the lock in `C`.

How to solve?

In xv6, we use defensive programming to avoid this problem:

1. When acquiring a lock with `acquire`, check if the current CPU is already holding this lock. If so, it's an AA-deadlock, and use `panic("already acquired by")` to halt kernel execution.

```
// Acquire the lock.
// Loops (spins) until the lock is acquired.
void acquire(spinlock_t *lk)
{
    uint64 ra = r_ra();
    push_off(); // disable interrupts to avoid deadlock.
    if (holding(lk))
        panic("already acquired by %p, now %p", lk->where, ra);

    // ...
}
```

2. For some frequently used functions, we use `assert(holding(&lk))` at the entry to assert that we hold this lock upon entering the function.

```
// vm.c

pte_t *walk(struct mm *mm, uint64 va, int alloc) {
    assert(holding(&mm->lock));
}

int mm_mappages(struct vma *vma) {
    // ...

    assert(holding(&vma->owner->lock));
}
```

ABBA-deadlock

ABBA deadlock is when thread 1 acquires locks in the order `A -> B`, and thread 2 acquires locks in the order `B -> A`. In a worst-case scenario, threads 1 and 2 each acquire A and B respectively, but then they are each waiting for B and A, and these two locks happen to be held by the other thread.

There are also more complex situations, such as:

- Thread 1 lock order: `A -> B`
- Thread 2: `B -> C`
- Thread 3: `C -> A`

We can summarize the **necessary conditions** for deadlock, treating locks (which can be generalized to "shared resources") as a ball:

1. Mutual Exclusion: A person gets a complete ball; it's impossible for two people to simultaneously get part of a ball.
2. Hold and wait: Holding the ball while waiting for additional balls.
3. No-Preemption: Cannot forcibly take the ball from someone else.
4. Circular wait: Forming a circular waiting relationship.

(Recommended reading: <https://dl.acm.org/doi/10.1145/356586.356588>)

Paper original text: This deadlock situation has arisen only because all of the following general conditions were operative: 1) Tasks claim exclusive control of the resources they require ("mutual exclusion" condition). 2) Tasks hold resources already allocated to them while waiting for additional resources ("wait for" condition). 3) Resources cannot be forcibly removed from the tasks holding them until the resources are used to completion ("no preemption" condition). 4) A circular chain of tasks exists, such that each task holds one or more resources that are being requested by the next task in the chain ("circular wait" condition).

Since these are **necessary conditions**, we only need to break any one of them to avoid the deadlock problem. In large systems, the last one is clearly the easiest to break.

Locking order can be seen as a directed edge, and the global locking order forms a graph (dependency graph). If there is a cycle in this graph, it indicates a deadlock problem.

How to avoid cycles: We can number the locks (Lock Ordering) and enforce acquiring them in increasing order.

Additionally, we can dynamically detect deadlocks.

DATA RACE

Concurrent bugs related to data races are mainly of the following two types:

1. Acquiring the wrong lock

```
void T_1() { spin_lock(&A); sum++; spin_unlock(&A); }
void T_2() { spin_lock(&B); sum++; spin_unlock(&B); }
```

1. Forgetting to acquire a lock

```
void T_1() { spin_lock(&A); sum++; spin_unlock(&A); }
void T_2() { sum++; }
```

1.3.3 User-Mode Locks

THE LOST WAKE-UP PROBLEM

In previous lessons, we discussed concurrency and synchronization code in kernel mode. So, how are mutual exclusion and synchronization implemented in user mode?

Shared Memory

The descriptions in this sub-section are based on the shared memory model. In Linux, these can be created using `mmap(2)` and `shm(2)`.

In xv6, our process model does not yet support shared memory, but the core idea is the same.

Differences between User Mode and Kernel

When we discuss concurrency, the most significant difference between user mode (usermode) and kernel mode (kernel) is that the kernel can mask interrupts, while user programs cannot.

In user mode, we can still use atomic instructions to implement `spinlock`. However, for `sleeplock`, things become more complex.

`sleeplock` requires the process to sleep when it cannot acquire the lock and be woken up when the lock is released. This means we need system calls to perform the "sleep" and "wake up" actions, because only the kernel can change a process's state.

```
void sleeplock_acquire(sleeplock_t *lk) {
  retry:
    if (__sync_lock_test_and_set(&lk->locked, 1) == 0) {
      // we succeed in 0-to-1 transition, aka, we get the lock.
      return;
    } else {
      // we fail to get the lock, sleep myself.
      int mypid = getpid();

      // append myself to the waiter queue.
      acquire(&lk->waiters_lock);
      append_waiter(lk, mypid);
      release(&lk->waiters_lock);
    }
  goto retry;
}
```

```

    // <-- this process may be wakeup here.

    syscall_sleep();

    // <-- then we will never be woken-up.

    goto retry;
}
}

void sleeplock_release(sleeplock_t *lk) {

    // A.
    __sync_lock_release(&lk->locked);

    // B.
    // find the next one to wakeup.
    acquire(&lk->waiters_lock);
    int nextone = pop_waiter(lk);
    release(&lk->waiters_lock);

    // C.
    syscall_wakeup(nextone);
}

```

Then, we will encounter the same "The Lost Wake-Up Problem" discussed in the Sync 1 lecture, where `syscall_sleep` might cause the wakeup signal to be lost.

The simplest correct approach is to place all locking and unlocking operations in the kernel. However, system calls are quite time-consuming, so we expect to keep the fast-path in user space. For `sleeplock`, the fast-path is when there is no contention for the lock, requiring only one atomic instruction to mark the lock as held.

FUTEX

Note

Please execute `man 7 futex` and `man 2 futex` in a Linux environment to view the relevant documentation.

`futex` is a syscall provided by the Linux kernel, used to implement fast user-space mutexes. Its definition is as follows:

```

long syscall(SYS_futex,
    uint32_t *uaddr, int futex_op, uint32_t val,
    const struct timespec *timeout, /* or: uint32_t val2 */
    uint32_t *uaddr2, uint32_t val3);

```

It accepts a virtual address `uaddr`, the user's expected value `val`, and the futex operation `futex_op`. `futex` distinguishes different futex objects based on the physical address behind the virtual address. Therefore, for `shm` across processes, as long as their respective virtual

addresses map to the same physical address, they can achieve synchronization through `futex`.

The two most important ops are:

1. `FUTEX_WAIT` : If the value at `uaddr` matches `val`, the thread sleeps, waiting for a `FUTEX_WAKE` operation on the futex word. If the kernel detects that the value at `uaddr` does not match the user's expected `val` (indicating a change in the context since the user called the `futex` syscall), it immediately returns `EAGAIN` from the system call without sleeping. (This avoids the lost-wakeup problem.)

The kernel also uses atomic instructions to access the physical address behind the virtual address, ensuring atomicity.

This operation tests that the value at the futex word pointed to by the address `uaddr` still contains the expected value `val`, and if so, then sleeps waiting for a `FUTEX_WAKE` operation on the futex word.

If the thread starts to sleep, it is considered a waiter on this futex word. **If the futex value does not match val, then the call fails immediately with the error EAGAIN.**

The purpose of the comparison with the expected value is to prevent lost wake-ups. If another thread changed the value of the futex word after the calling thread decided to block based on the prior value, and if the other thread executed a `FUTEX_WAKE` operation after the value change and before this `FUTEX_WAIT` operation, then the calling thread will observe the value change and will not start to sleep.

2. `FUTEX_WAKE` : Wake up all waiters waiting on `uaddr` .

This operation wakes at most `val` of the waiters that are waiting (e.g., inside `FUTEX_WAIT`) on the futex word at the address `uaddr`. Most commonly, `val` is specified as either 1 (wake up a single waiter) or `INT_MAX` (wake up all waiters).

`PTHREAD_MUTEX_T`

`pthread_mutex_t` is provided by the pthread library in libc, and it uses the futex syscall to implement **mutex** functionality.

We can explore how `pthread_mutex_t` is implemented:

glibc Source Code

glibc source code is very complex and highly optimized for performance.

This part of the code is located in `nptl/lowlevellock.c` and `sysdeps/nptl/lowlevellock.h`.

The basic functionality of `pthread_mutex_t` relies on `lll_lowlevellock`.

`pthread_mutex_t` has a `uint32_t` which represents the lock state: 0 means unlocked, 1 means locked but no waiters, and `>1` means locked but potentially with waiters.

```
void lll_lock(uint32_t* futex) {
    // try to make futex transit from 0 (UNLOCKED) to 1 (LOCKED).
    if (atomic_compare_and_exchange_bool_acq(futex, 1, 0)) {
        // if cmpxchg fails:

        // try to exchange 2 into `*futex`, return the original value.
        while (atomic_exchange_acquire (futex, 2) != 0) {

            // if old value is not `UNLOCKED`
            syscall_futex(futex, FUTEX_WAIT, 2); /* Wait if *futex == 2. */

            // if syscall returns, *futex is not 2 or someone wakes me up.
        }

        // when we get there, old *futex is 0, now 2 (LOCKED).
    }
    // succeed, LOCKED (*futex is 1 or 2)
}

void lll_unlock(uint32_t* futex) {
    // exchange 0 UNLOCKED into futex
    int __oldval = atomic_exchange_release (futex, 0);

    if (__oldval > 1) {
        // wake up one waiter
        syscall_futex(futex, FUTEX_WAKE, 1);
    }
}
```

Locking steps:

1. First, try to transition futex from `0->1`. If this fails, it means another thread holds the lock.
2. Then, continuously loop attempting to transition futex from `0->2`. If successful, the lock is acquired.
3. Otherwise, wait on `futex` using the `futex_wait` system call. When returning from the syscall, retry step 2.


```

      |           lock (0->2, fail)           |
      |           |                           |
unlock(store 0, see 2)      |           |
      |           |                           | | |
      |           |           futex_wait()     |
      |           |           |               |
      |           |           |           (EAGAIN, kernel sees 0) |
      |           |           |           |   (woken-up)         |
      |           |           |           |                       |
      |           |           |           |           (released)  |

```

1.3.4 Sync 1 & 2 Lab Exercise Analysis

Will be discussed in class.